Qualité de développement

Nicolas Ferry
Université Côte d'Azur – IUT Informatique – BUT 1
2025 - 2026

Gestion de version

Et introduction à Git

VCS: Principes et définitions

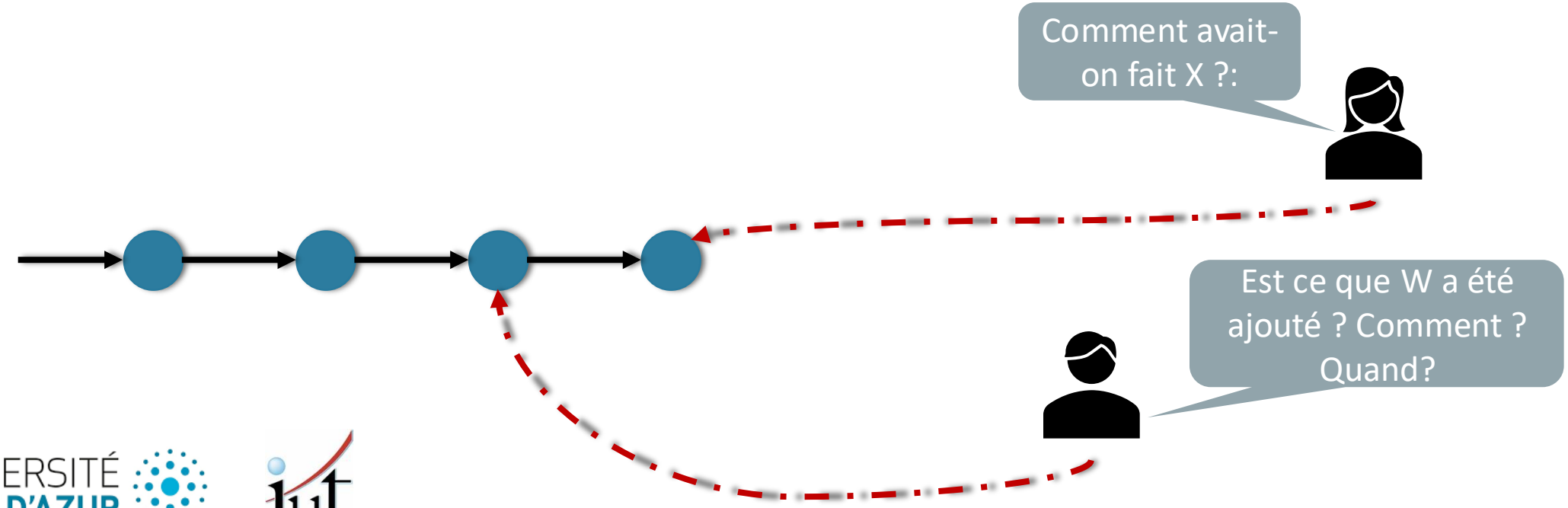
- Système de **contrôle de version** (Version Control System - VCS):
  - Outil **pour collaborer**, enregistrer et retrouver différentes versions d'un projet.
  - A l'origine dédié au source code.

- Définitions :
  - **Révision :** Contenu d'un projet à un moment de son cycle de vie.
  - **Dépôt** (repository) : Le projet et tout son historique (toutes ses versions et variantes).
  - **Branche** (branch) : Une variante du projet.

➢ On enregistre des **révisions** d'un projet dans le **dépôt** afin de pouvoir les retrouver plus tard.

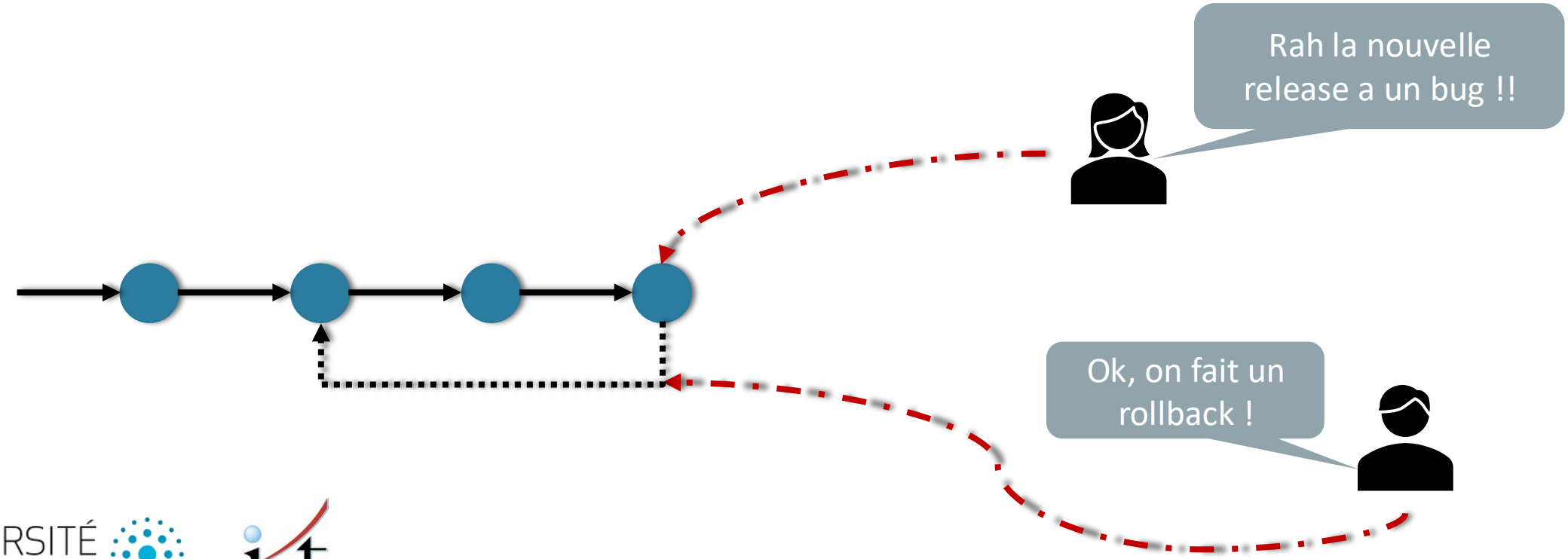
Pourquoi les VCS

- **Pour garder un historique des révisions.**
  - Enregistrer une nouvelle version.
  - Accéder à une révision.
  - Ajouter des informations décrivant les révisions (e.g., pourquoi on arrive à cette révision, sa particularité).



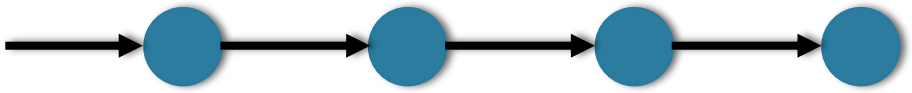
Pourquoi les VCS

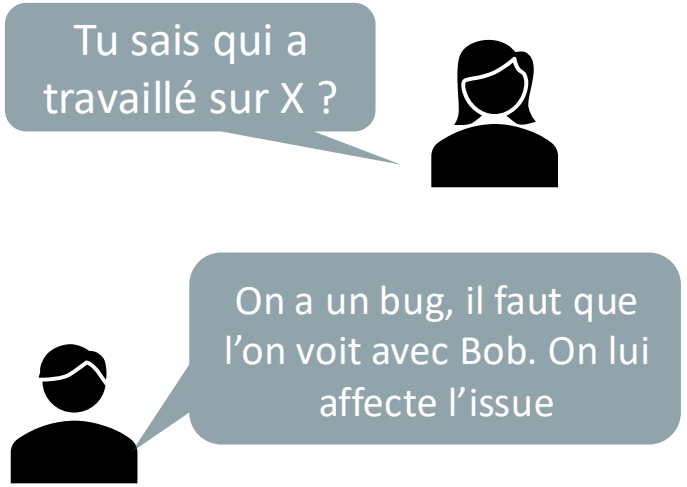
- Pour garder un historique des révisions.
- **Pour pouvoir faire des “rollbacks”.**



Pourquoi les VCS

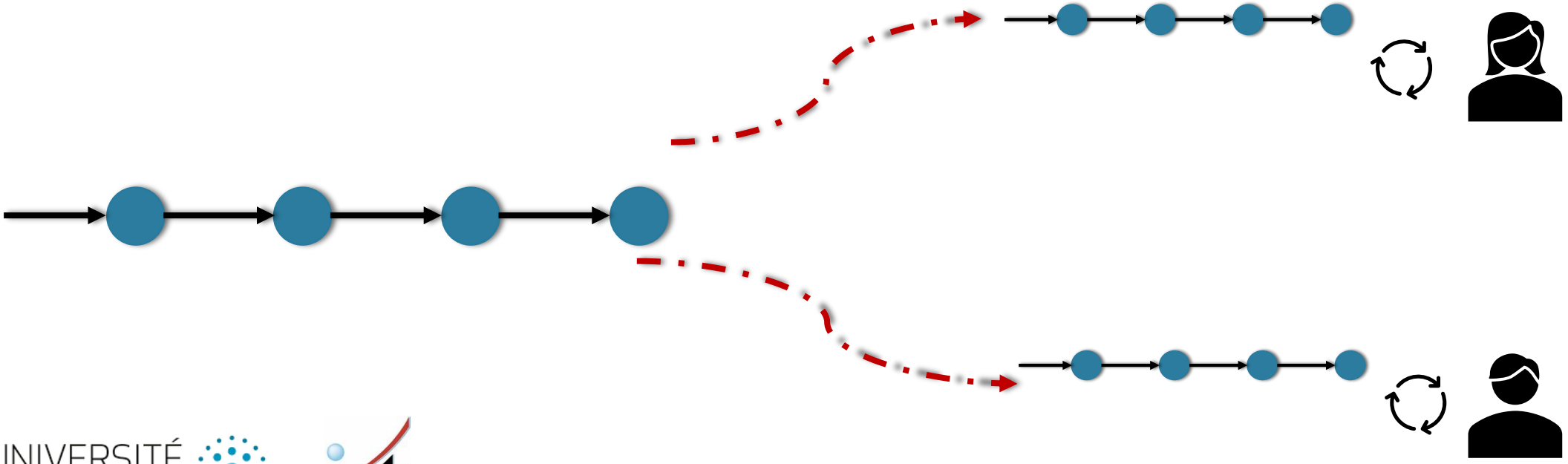
- Pour garder un historique des versions.
- Pour pouvoir faire des “rollback”.
- **Pour savoir qui a travaillé sur quoi.**
  - Associer un responsable à chaque ligne de code.
    - Pas pour blâmer quelqu’un !
    - Mais savoir qui a l’expertise, introduit un bug et pour quelle raison.





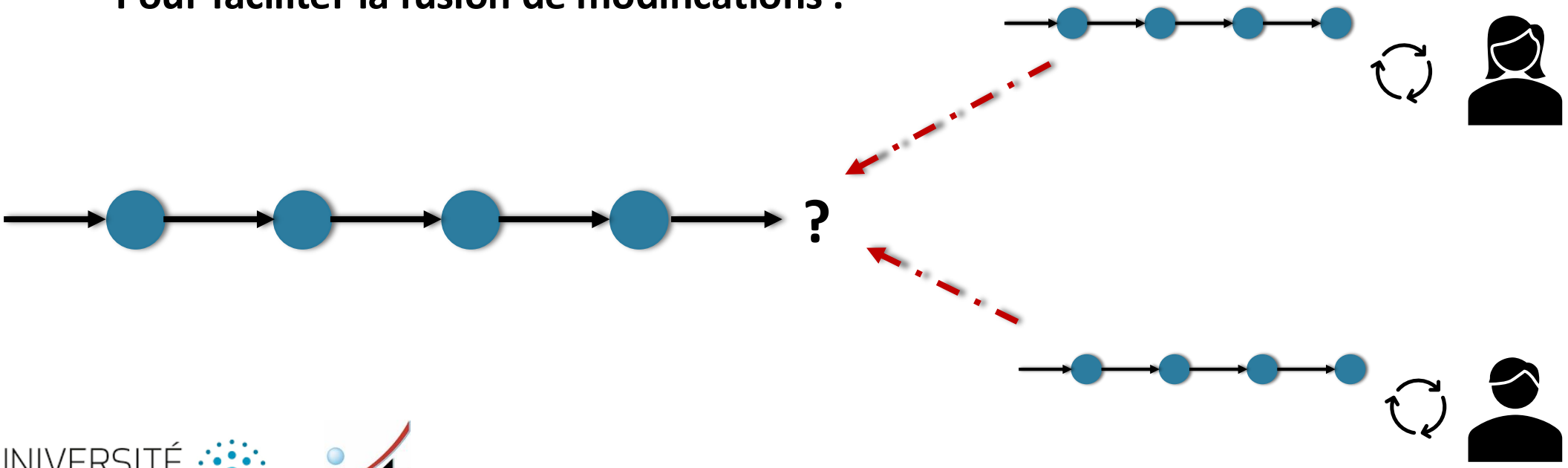
Pourquoi les VCS

- Pour garder un historique des versions.
- Pour pouvoir faire des “rollback”.
- Pour savoir qui a travaillé sur quoi.
- **Pour partager du code et des modifications.**



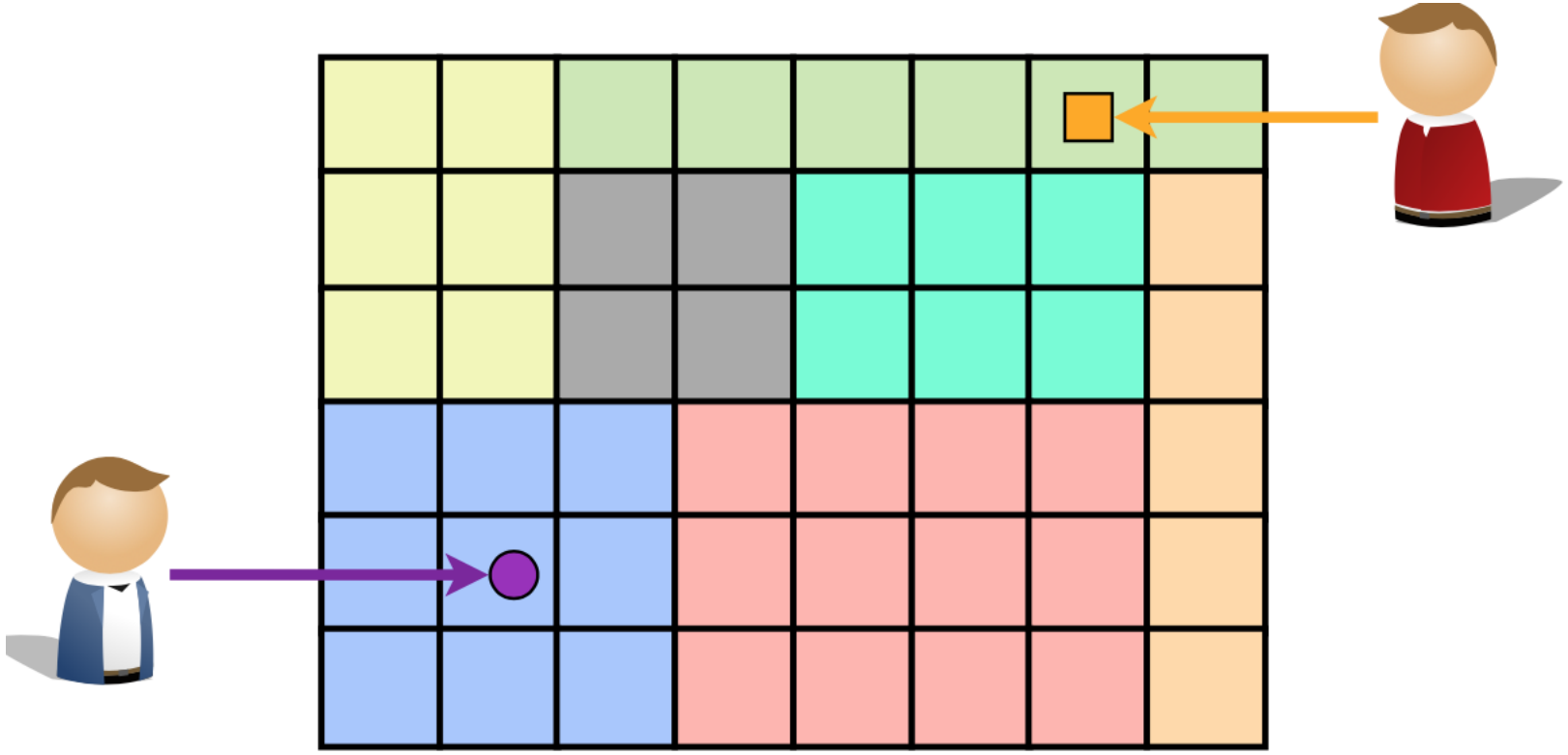
Pourquoi les VCS

- Pour garder un historique des versions.
- Pour pouvoir faire des “rollback”.
- Pour savoir qui a travaillé sur quoi.
- Pour partager du code et collaborer.
- **Pour faciliter la fusion de modifications .**



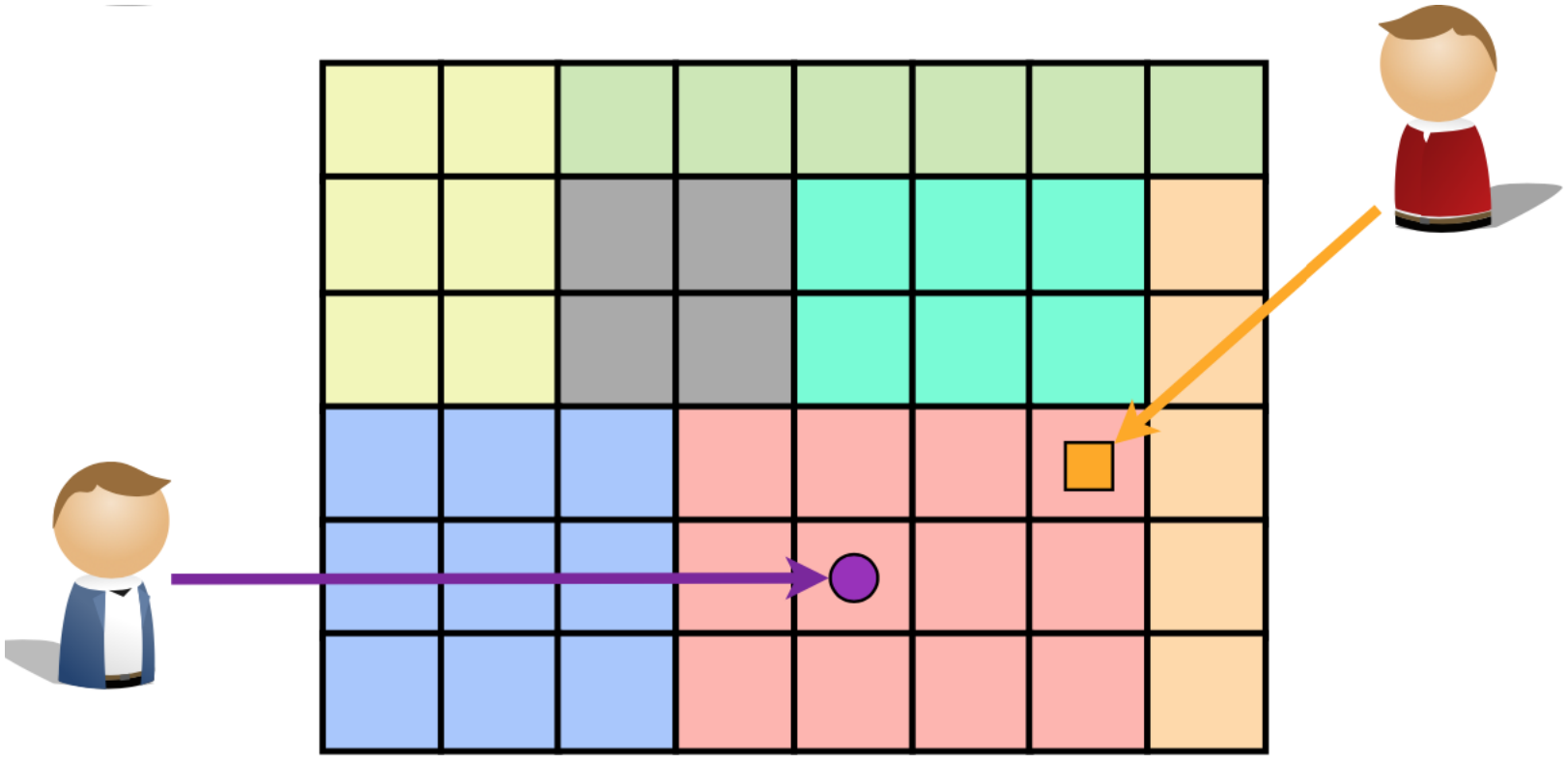
UNIVERSITÉ CÔTE D'AZUR
iut Nice Côte d'Azur

Fusion – sans problèmes



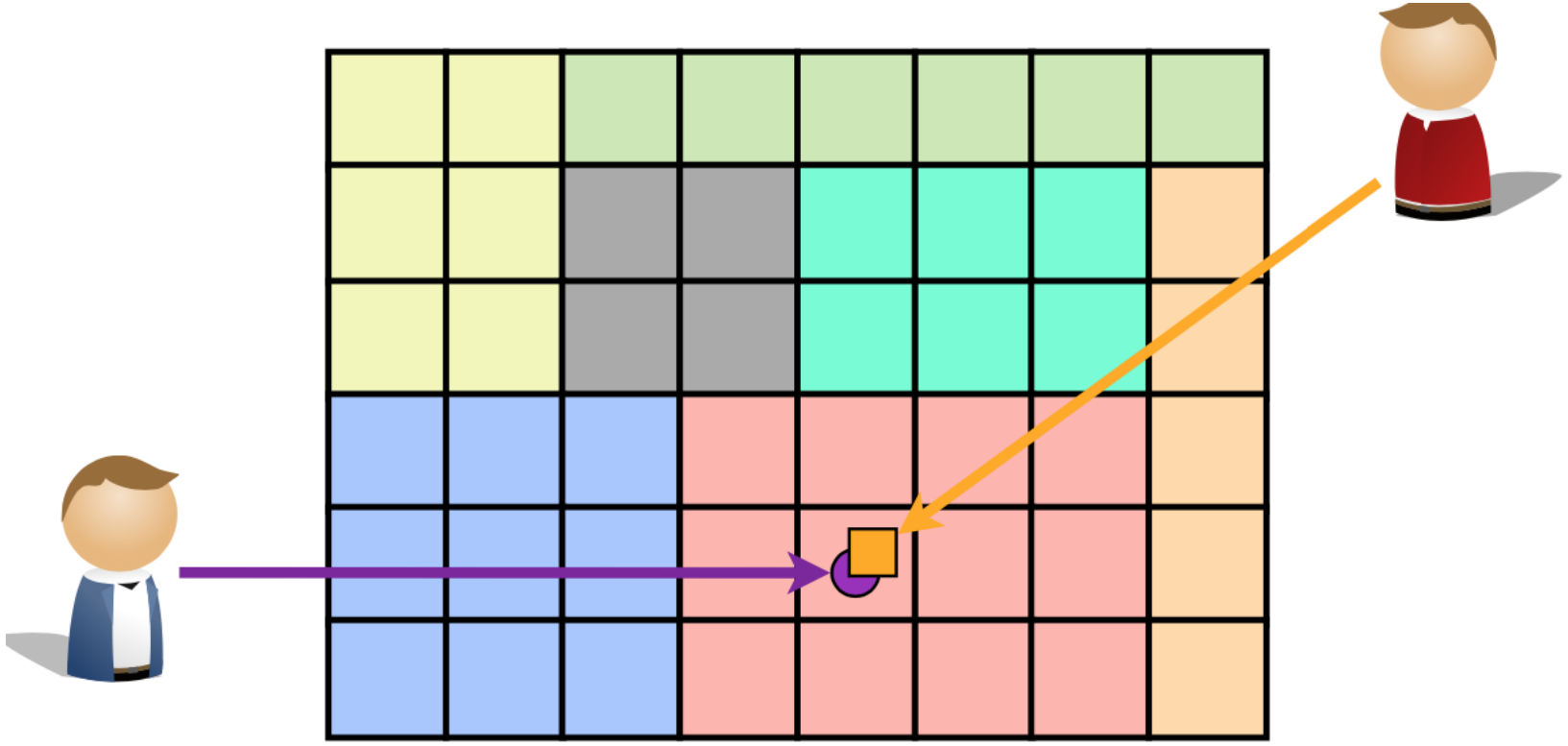
Source : Sébastien Mosser

Fusion automatique



Source : Sébastien Mosser

Conflit de fusion

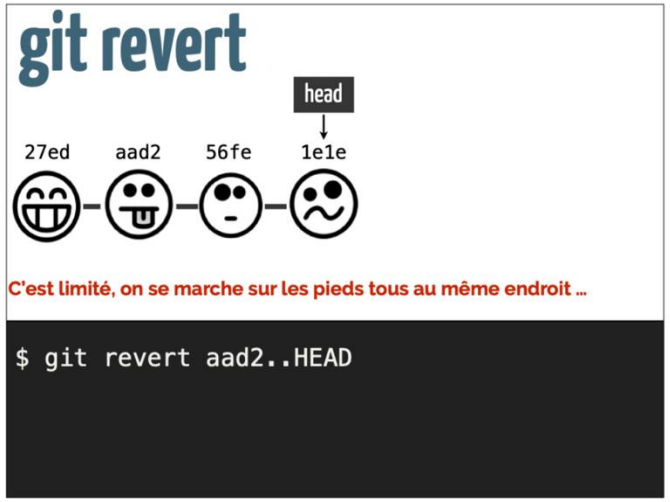


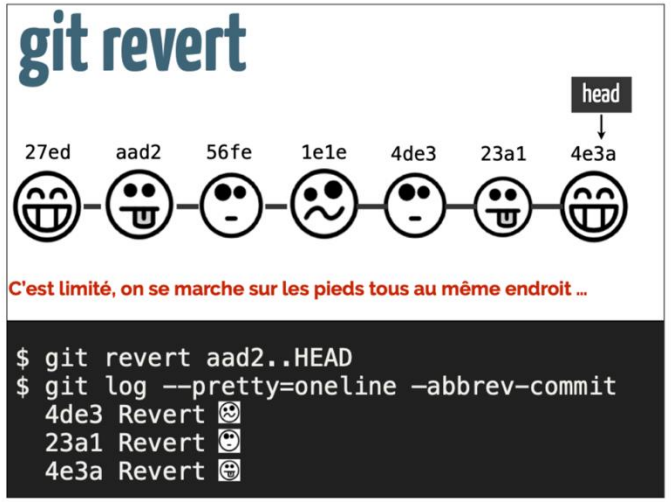
Source : Sébastien Mosser

Pourquoi les VCS

- Pour garder un historique des révisions.
- Pour pouvoir faire des “rollback”.
- Pour savoir qui a travaillé sur quoi.
- Pour partager du code et collaborer.
- Pour faciliter la fusion de modifications .
- **Pour avoir plusieurs variantes.**

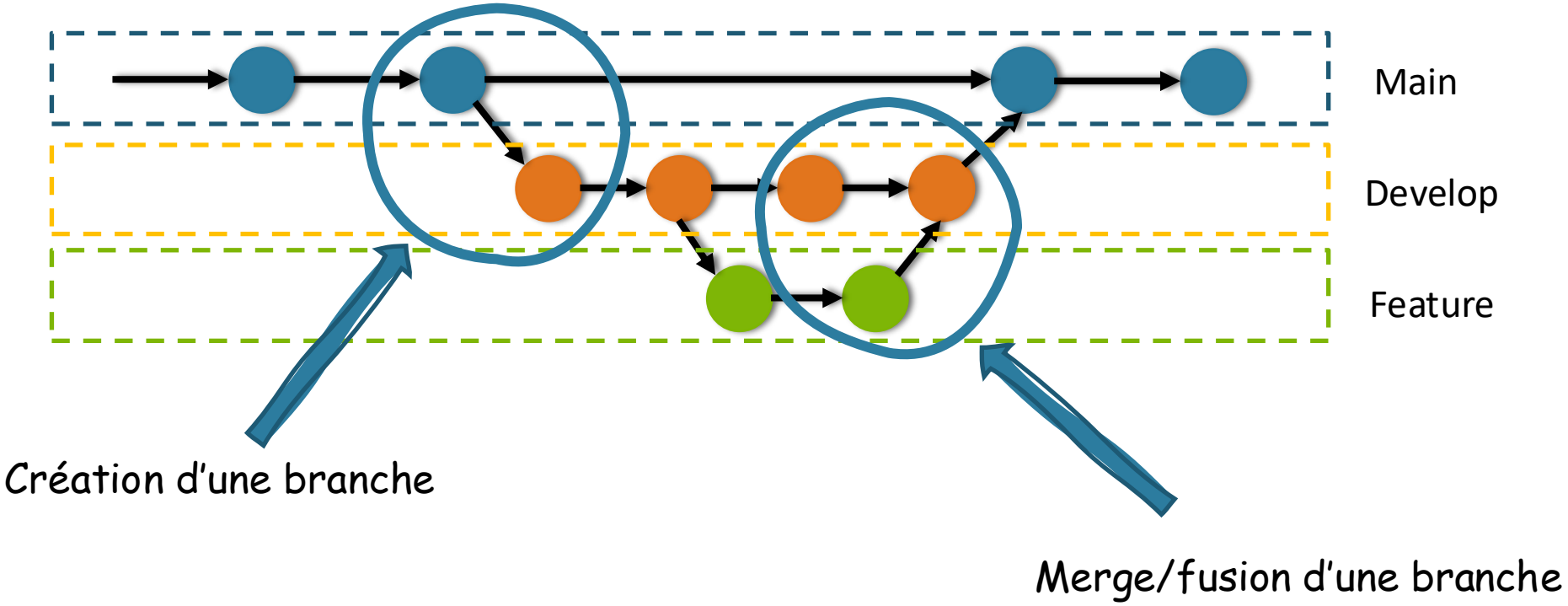
Bof





https://github.com/ace-lectures/H20-MGL-7460/blob/master/cours/2_versions_tests.pdf

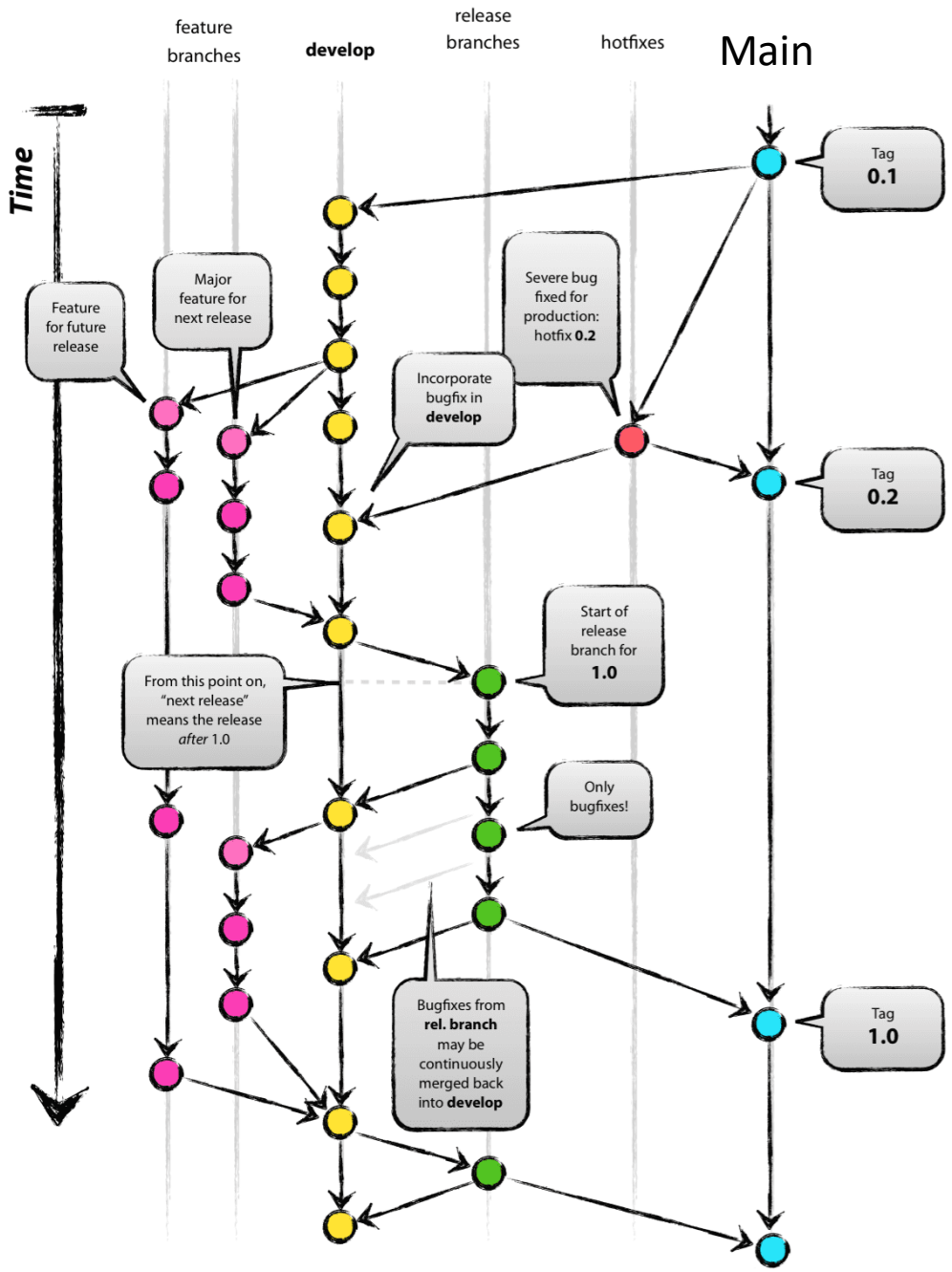
Concept de branches



On contrôle quand on diverge et converge

Gestion des branches

- Le modèle **Git-Flow.**
- Un peu complexe, une variante plus simple pour petit projets :
  - 2 branches tout le long du projet:
    - Main
    - Develop
  - Une branche par feature/bug fix
    - Créé à partir de develop
    - On travaille dessus
    - On la merge dans develop
    - Si testé valide on l'intègre dans Main.
- Ou encore plus simple : https://guides.github.com/introduction/flow/



Source: https://nvie.com/posts/a-successful-git-branching-model/

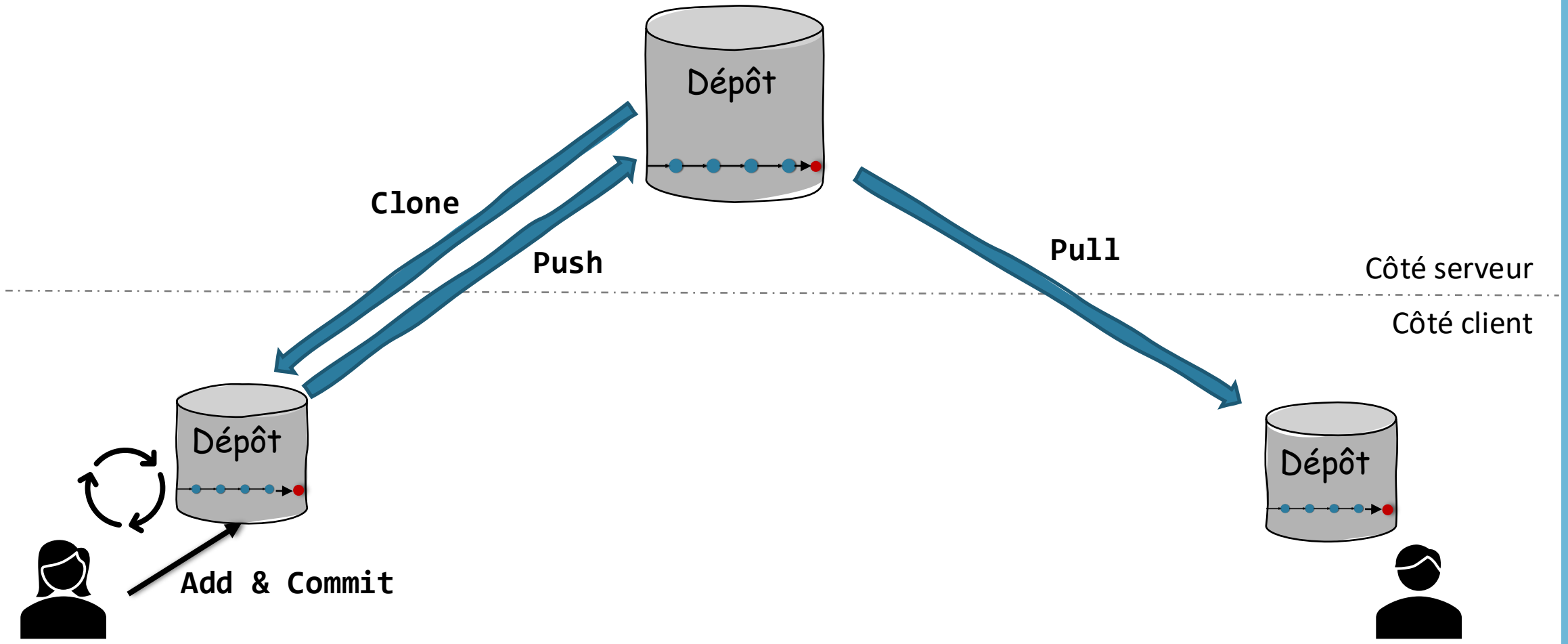
Rappel

On ne stocke pas d'informations personnelles dans un dépôt !!!

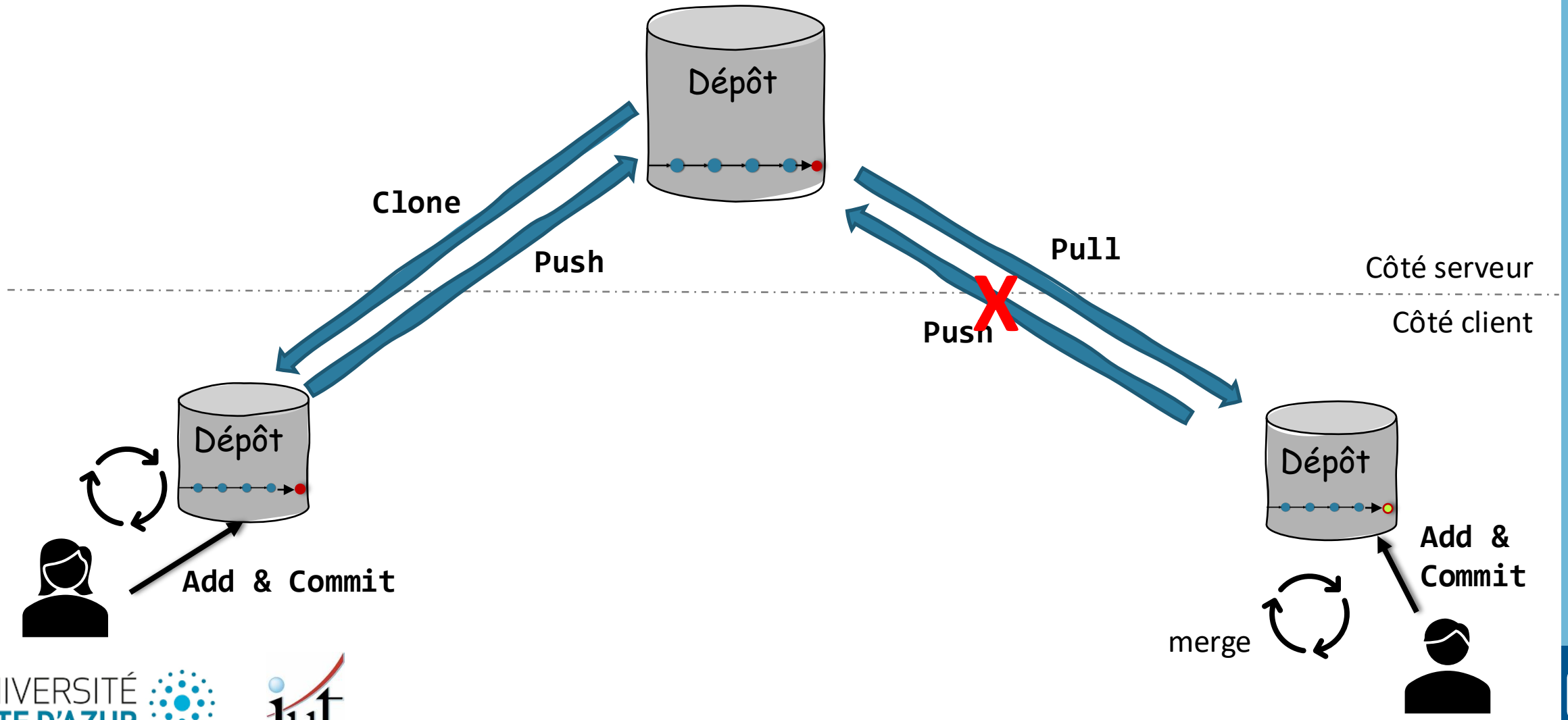
Git

- Un système de contrôle de versions **décentralisé** (tout comme Bazaar, Mercurial).
  - Les dépôts existent coté client et coté serveur.
  - Connexions de type pair à pair.
- Créé par Linus Torvalds en 2005.
- Licence GPL v2.

Modèle décentralisé



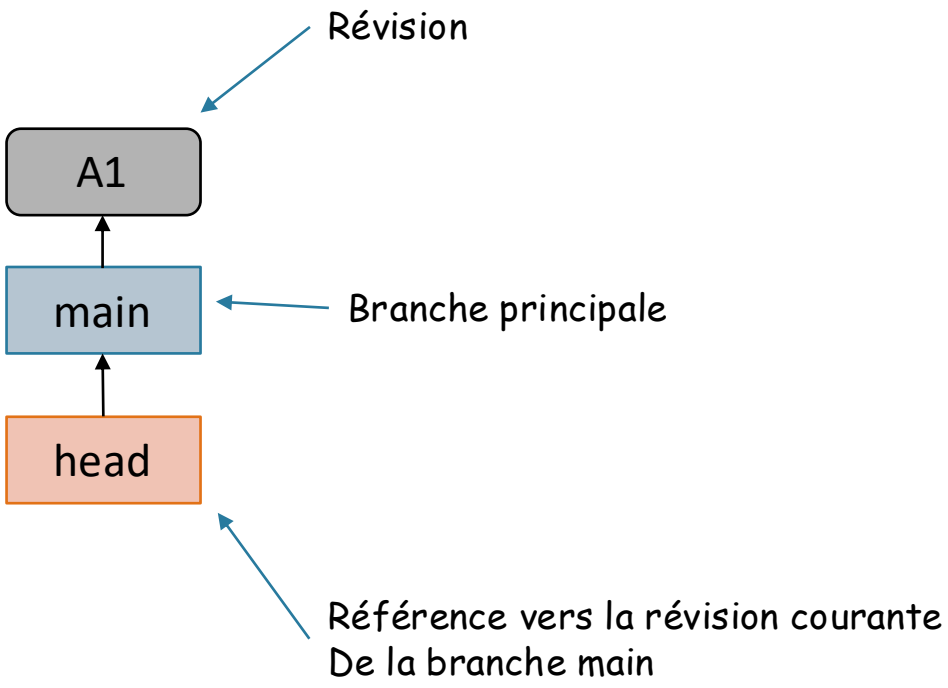
Processus avec conflits

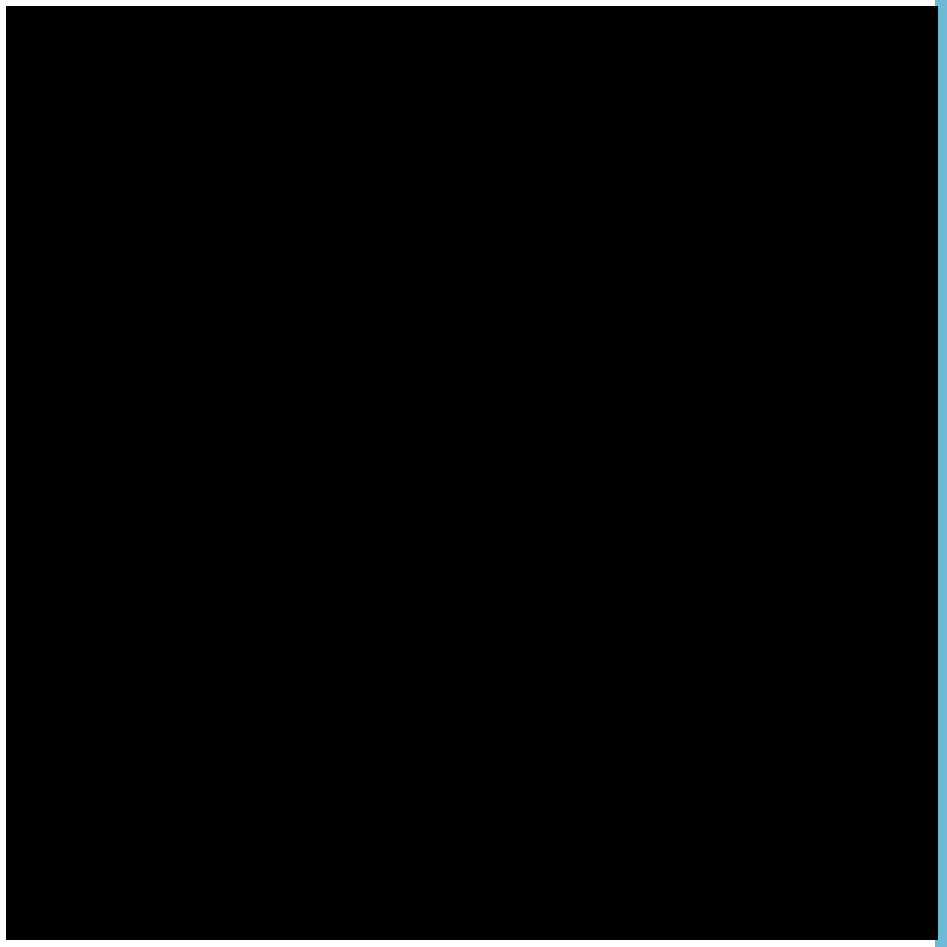


Récapitulatif

- Un ensemble de commandes pour la manipulation du code :
  - **`clone`** : récupérer une copie du code
  - **`commit`** : pour enregistrer ses modifications
  - **`push`** : pour partager ses modifications
  - **`pull`** : pour récupérer les modifications des autres

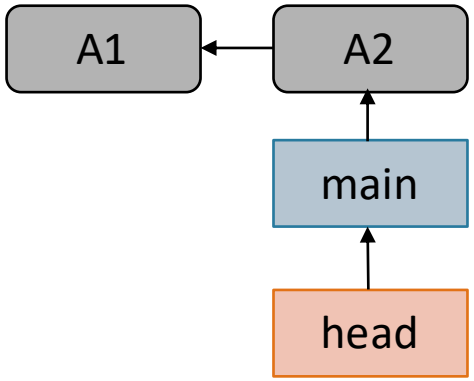
Git et branches





Source : https://github.com/ace-lectures/H20-MGL-7460/blob/master/cours/2_versions_tests.pdf

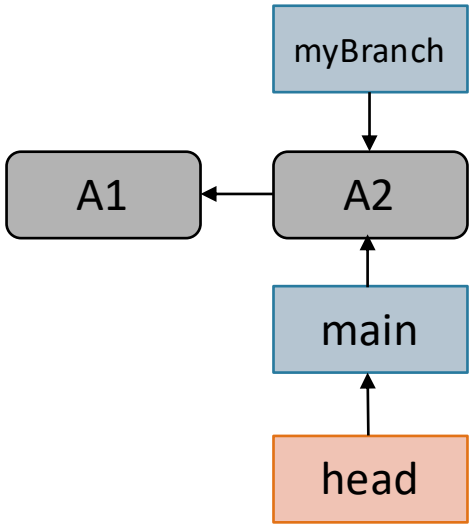
Git et branches




```
> git commit –m “…” -a
```


Source : https://github.com/ace-lectures/H20-MGL-7460/blob/master/cours/2_versions_tests.pdf

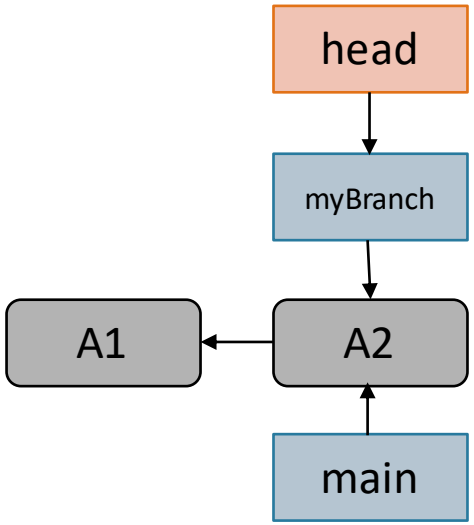
Git et branches




```
> git commit -m "…" -a
> git branch myBranch
```


Source : https://github.com/ace-lectures/H20-MGL-7460/blob/master/cours/2_versions_tests.pdf

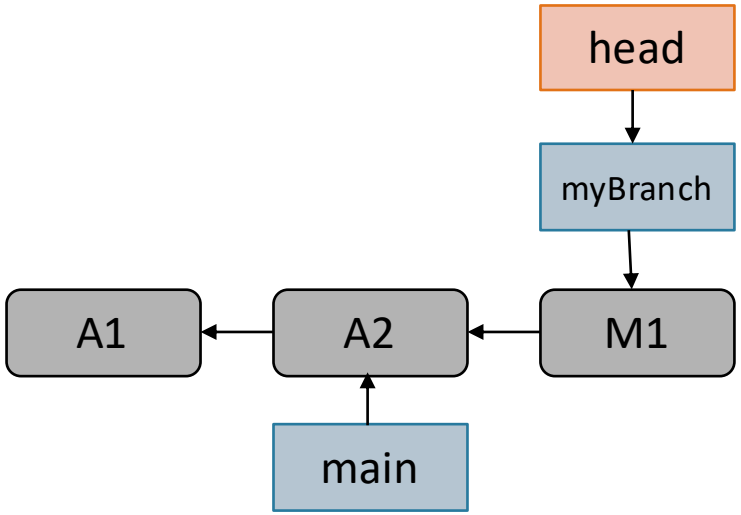
Git et branches




```
> git commit -m "…" -a
> git branch myBranch
> git checkout myBranch
```


Source : https://github.com/ace-lectures/H20-MGL-7460/blob/master/cours/2_versions_tests.pdf

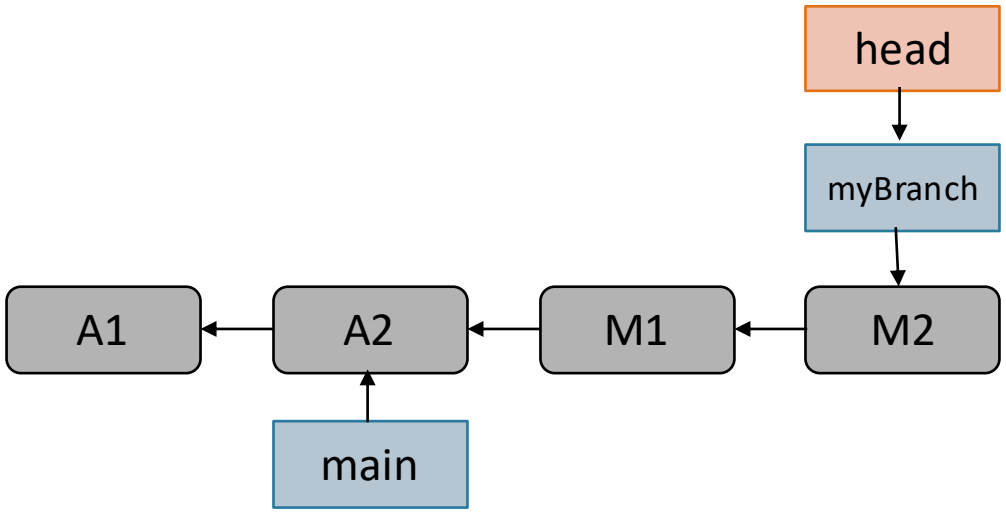
Git et branches




```
> git commit –m “…” –a
> git branch myBranch
> git checkout myBranch
> git  commit –m “…” -a
```

Source : https://github.com/ace-lectures/H20-MGL-7460/blob/master/cours/2_versions_tests.pdf

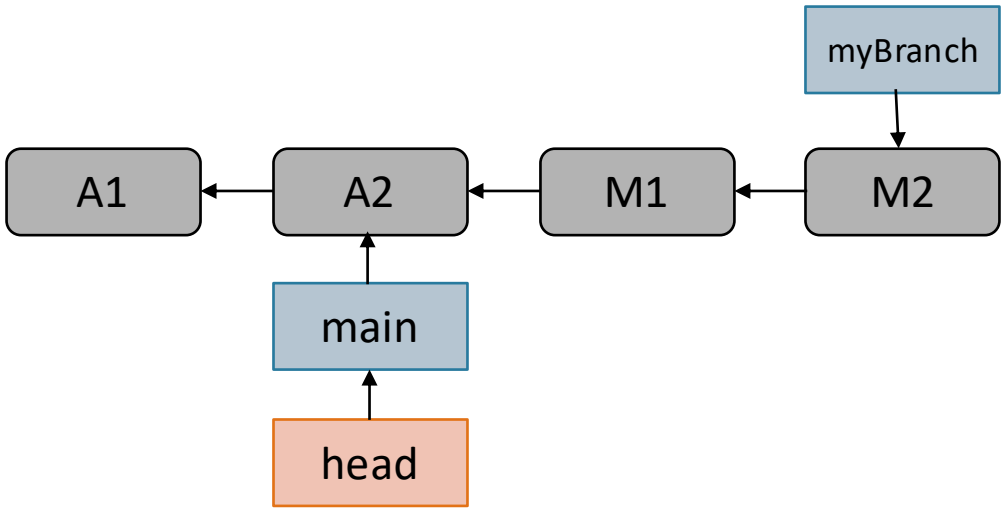
Git et branches




```
> git commit –m “…” –a
> git branch myBranch
> git checkout myBranch
> git  commit –m “…” -a
> git  commit –m “…” -a
```

Source : https://github.com/ace-lectures/H20-MGL-7460/blob/master/cours/2_versions_tests.pdf

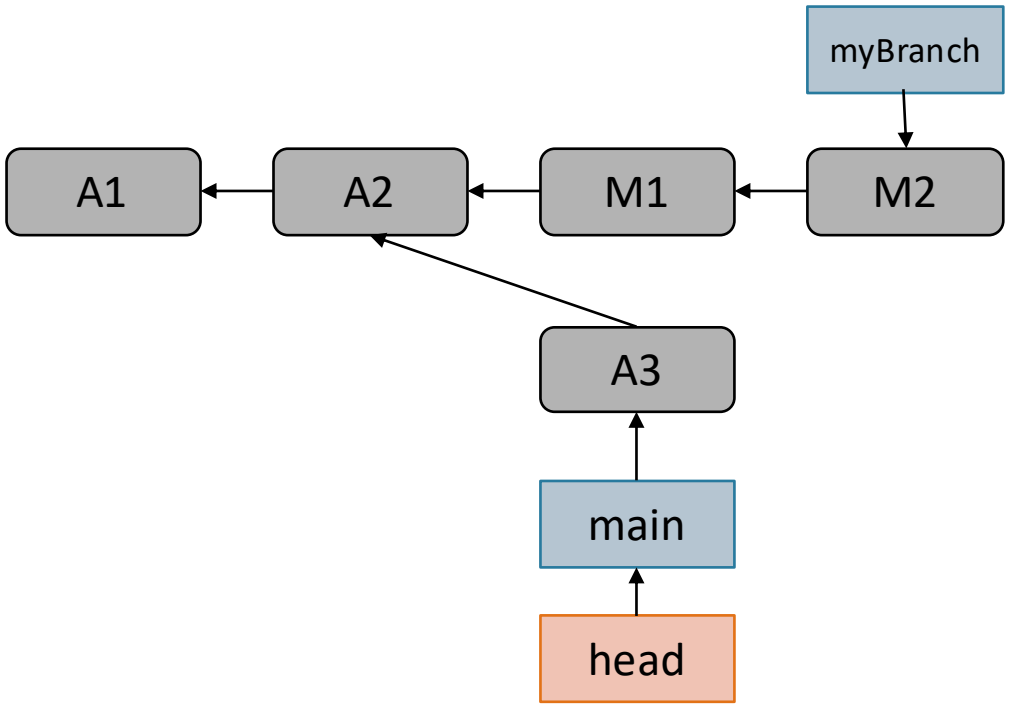
Git et branches




```
> git commit –m “…” –a
> git branch myBranch
> git checkout myBranch
> git  commit –m “…” -a
> git  commit –m “…” –a
> git checkout main
```

Source : https://github.com/ace-lectures/H20-MGL-7460/blob/master/cours/2_versions_tests.pdf

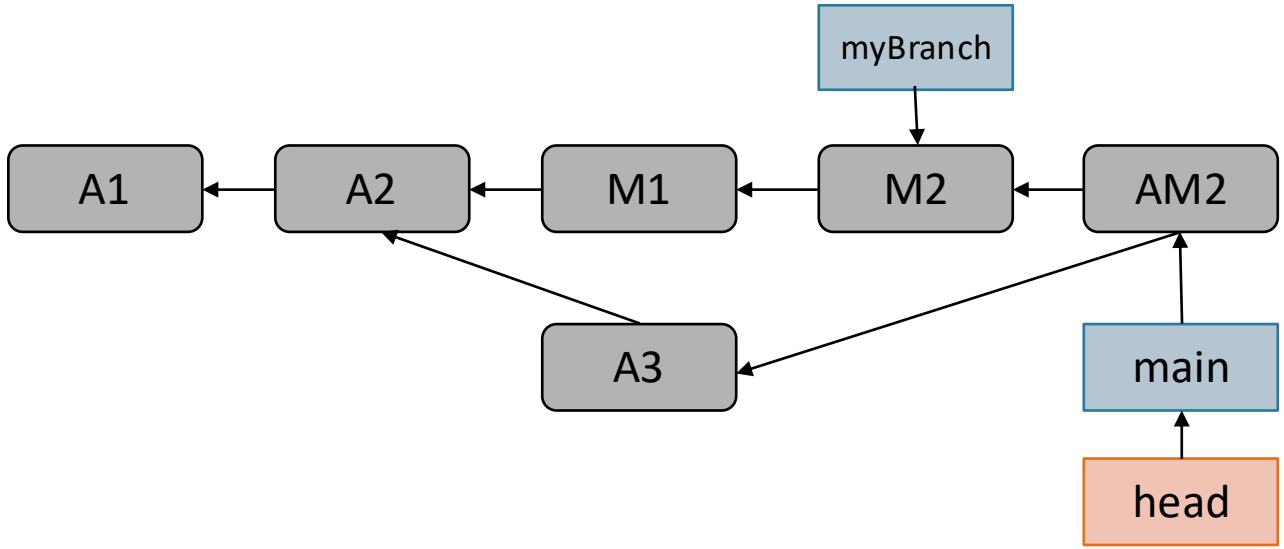
Git et branches




```
> git commit –m “…” –a
> git branch myBranch
> git checkout myBranch
> git  commit –m “…” -a
> git  commit –m “…” -a
> git checkout main
> git  commit –m “…” -a
```

Source : https://github.com/ace-lectures/H20-MGL-7460/blob/master/cours/2_versions_tests.pdf

Git et branches

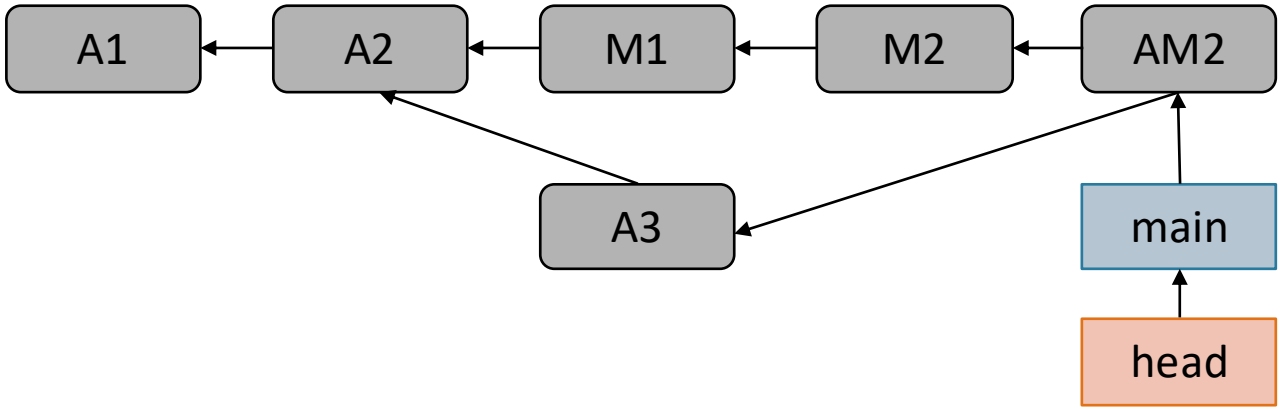



```
> git commit –m “…” –a
> git branch myBranch
> git checkout myBranch
> git  commit –m “…” -a
> git  commit –m “…” -a
> git checkout main
> git  commit –m “…” –a
> git merge myBranch
```

** If Master has not diverged, instead of creating a new commit, git will just point master to the latest commit of the feature branch.*
There won't be any "merge commit" in fast-forwarding merge.

Source : https://github.com/ace-lectures/H20-MGL-7460/blob/master/cours/2_versions_tests.pdf

Git et branches




```
> git commit –m “…” –a
> git branch myBranch
> git checkout myBranch
> git  commit –m “…” -a
> git  commit –m “…” -a
> git checkout main
> git  commit –m “…” –a
> git merge myBranch
> git branch –d myBranch
```

Source : https://github.com/ace-lectures/H20-MGL-7460/blob/master/cours/2_versions_tests.pdf

En cas de conflits !

```
<<<< HEAD
Hello the world
=======
Hello world
>>>> Branch-a
```

Ce qui se trouve dans le dépôt

Ce qui vous voulez pousser

- Un fichier qui typiquement ne compile !
- Faites le merge manuellement et retirez le texte en trop `git commit -am`.

Pour commencer

1. Créer un dépôt :
   `git init`
2. Ajouter des fichiers :
   `git add .`
3. Vérifier l'état courant :
   `git status`
4. Valider les changements :
   `git commit  –m "initialisation du depot avec README et license"`
5. Déclare le dépôt distant :
   `git remote add origin` git@github.com:lp-dam/first_app.git
6. On pousse :
   `git push`

Si on part d'un dépôt existant : `git clone <depot>`

Ensuite

- Vérifier le status de la branche courante :
  `git checkout`
- Créer la branche “develop” :
  `git checkout –b develop`
- Regarder la liste des branches existantes :
  `git branch`
- Après developpement on ajoute et commit (dans la branche develop):
  `git commit –am “some work” mon_fichier`
- Encore du developpement:
  `git commit –m “some more” mon_fichier`
- Oublie d’un fichier :
  `git add fichier_oublié`
- On amende :
  `git commit --amend`

Ensuite 2

- Retourner sur la branche master :
  `git checkout main`
- On merge develop avec master :
  `git merge develop`
- On supprime develop? :
  `git branch –d develop`
- On synchronize :
  `git push`

Créer des releases

- On tag un commit :
  `git tag –a v2.0 ssdr9 –m “Second release with …”`
- On pousse le tag:
  `git push origin v2.0`
- On regarde les details du tag :
  `git show v2.0`

Et GitHub/GitLab dans tout ça ?

- Des applications web pour gérer des dépôts côté serveur.
- i.e., Gère des dépôts git et offre des interfaces pour:
  - Création suppression de dépôts, y compris leur accès (publique, privé, qui ?)
  - Visualisation l'historique d'un projet
  - Communication et organisation du projet (issues, Kanban, etc.)
  - Documentation des projets (wiki, etc.)
  - Continuous delivery?
  - Etc.